Table 1. Evaluation results of models across different settings and benchmarks. LC and WR refer to length-controlled and raw win rates, respectively. The SFT model is equivalent to the CLM when λ is set to 0.

Method	Llama3.2-3B-Base					Llama3.1-8B-Base				
	AlpacaEval2		Arena-Hard	MT-Bench		AlpacaEval2		Arena-Hard	MT-Bench	
	LC (%)	WR (%)	WR (%)	1-turn	2-turn	LC (%)	WR (%)	WR (%)	1-turn	2-turn
SFT	2.83	2.42	2.00	6.39	5.53	3.06	2.79	3.60	6.99	6.43
DPO_{full}	11.44	10.47	11.60	6.98	6.51	20.16	15.02	26.20	7.66	7.28
$\text{CLM}_{\lambda=0.5}$	7.28	6.62	6.90	6.66	6.18	12.19	9.80	14.00	7.26	7.03
$\text{CLM}_{\lambda=1.0}$	11.53	11.93	11.00	**7.11**	6.36	20.45	17.86	24.40	7.46	7.13
$\text{CLM}_{\lambda=1.5}$	**12.71**	14.84	17.30	6.88	**6.74**	**21.99**	20.81	29.80	**7.67**	**7.31**
$\text{CLM}_{\lambda=2.0}$	12.01	**14.92**	**21.80**	6.93	6.24	20.90	**21.09**	**34.30**	7.36	7.04
Method	Qwen2.5-1.5B-Base					Qwen2.5-7B-Base				
	AlpacaEval2		Arena-Hard	MT-Bench		AlpacaEval2		Arena-Hard	MT-Bench	
	LC (%)	WR (%)	WR (%)	1-turn	2-turn	LC (%)	WR (%)	WR (%)	1-turn	2-turn
SFT	2.55	2.54	2.80	6.85	5.30	5.42	3.59	8.30	7.43	7.01
DPO_{full}	8.38	8.36	15.60	7.49	6.68	25.20	20.92	45.50	8.21	**7.80**
$\text{CLM}_{\lambda=0.5}$	4.75	5.04	7.00	6.98	6.13	14.19	11.52	30.50	8.17	7.53
$\text{CLM}_{\lambda=1.0}$	**9.13**	10.50	14.80	7.51	6.59	25.74	25.83	45.10	8.21	6.70
$\text{CLM}_{\lambda=1.5}$	8.64	**11.23**	**16.30**	7.48	6.54	30.69	34.15	**50.80**	8.48	5.94
$\text{CLM}_{\lambda=2.0}$	7.15	10.38	15.30	**7.58**	**6.68**	**30.99**	**35.35**	50.30	**8.51**	5.58

used by the community (Meng et al., 2024). All these benchmarks are auto-evaluated using LLMs. And we use Qwen2.5-72B-Instruct (Yang et al., 2024a) as the backend API to provide judgment. We use the zero-shot setting to test the reasoning ability across four benchmarks, including MMLU (Hendrycks et al., 2021), CMMLU (Li et al., 2024a), Truthful-QA (Lin et al., 2021), and GSM8K (Cobbe et al., 2021). We evaluate these benchmarks using *llm-evaluation-harness* (Gao et al., 2024) repo.

Training details The training framework utilizes the LLaMA-Factory (Zheng et al., 2024b) repository. All training processes involve full fine-tuning over one epoch with a warm-up ratio of 0.1. The batch size is set to 128. For SFT training, a learning rate of 2e-6 is used for all models. For the DPO-trained model, different learning rates and β values are explored, and the best-performing configuration is selected.

Inference details All inferences are conducted using the vLLM engine (Kwon et al., 2023) with a temperature setting of 0.0 (**greedy decoding**) and a maximum generation length of 4096 tokens.

AlpacaEval2 and Arena-hard are designed to evaluate the quality and relevance of model responses to given instructions, aiming to assign higher scores to human-preferred answers (Dubois et al., 2024; Li et al., 2024b). MT-bench is a multi-turn question set to assess a model's ability to engage in multi-turn conversations and effectively follow instructions (Zheng et al., 2023).

5.1. Performance of Alignment

As presented in Table 1, the SFT model demonstrates strong instruction-following capabilities, as evidenced by its MT-Bench scores. However, its alignment performance remains limited. To address this, we employ the DPO algorithm to fine-tune the SFT models fully, significantly enhancing their alignment performance. Furthermore, we evaluate our proposed CLM method with $\lambda = 1.0$. The results indicate that the alignment performance of CLM is on par with that of the fully fine-tuned DPO model. However, all MT-Bench results reveal a decline in the model's second-round conversational abilities.

5.2. Interpolation and Extrapolation of Alignment

Table 1 summarizes the performance of various models across different benchmarks. The results demonstrate that preference learning enhances the models' alignment performance. However, as alignment improves, we observe fluctuations in performance across the benchmarks. This variability stems from each benchmark employing distinct evaluation criteria to capture specific human values or abilities. Furthermore, these scores do not always increase, as they are constrained by the size of the model's parameters.

Table 2. Evaluation results of models across different benchmarks. We evaluate these benchmarks using *llm-evaluation-harness* (Gao et al., 2024) repo.

Method	Llama3.1-8B-Base				Qwen2.5-7B-Base			
	MMLU	CMMLU	GSM8K	Truthful-QA	MMLU	CMMLU	GSM8K	Truthful-QA
$\text{CLM}_{\lambda=0}$	59.78	47.56	57.01	54.29	**71.39**	**81.84**	**82.22**	56.37
$\text{CLM}_{\lambda=0.5}$	59.80	47.74	60.05	54.83	70.80	81.53	77.75	57.26
$\text{CLM}_{\lambda=1.0}$	59.91	47.86	**60.65**	56.90	70.37	81.28	72.71	58.44
$\text{CLM}_{\lambda=1.5}$	**60.04**	47.92	57.92	59.98	70.03	80.94	68.99	59.59
$\text{CLM}_{\lambda=2.0}$	60.01	**48.04**	54.74	**61.07**	69.31	80.49	65.50	**61.05**

We make the following observations:

There is a clear trend that CLM can implement alignment interpolation and extrapolation. Taking the Arena-hard benchmark as an example, when $\lambda = 0.5$, the alignment strength lies between the SFT model and the CLM model with $\lambda = 1.0$. By appropriately increasing the value of λ, the alignment ability can be further enhanced, even surpassing the performance of the DPO fully fine-tuned model. **In this case**, it suggests that when $0 < \lambda < 1$, the alignment ability is interpolated, while for $\lambda > 1$, the alignment ability is extrapolated. A similar phenomenon is observed for AlpacaEval 2 and the first-turn dialogue in MT-Bench.

The absence of alignment extrapolation phenomenon in the second-turn dialogue when $\lambda > 1$. As shown in Table 1, the Qwen2.5-7B-Base model exhibits an inverse trend: the MT-Bench score decreases as λ increases in the second-turn dialogue. However, **when $\boldsymbol{\lambda = 0.5}$**, the model achieves a score of 7.53, surpassing both the SFT model (score: 7.01) and the CLM model with $\lambda = 1.0$ (score: 6.70), **demonstrating the extrapolation phenomenon.** We attribute this behavior to preference learning. The additional layer was fine-tuned on the Ultrafedback preference dataset, composed solely of single-turn dialogues (Cui et al., 2024). As a result, increasing λ enhances the model's ability to adhere to single instructions (overfitting to the reward). Still, this improvement comes at the cost of its performance in multi-turn dialogues, especially in maintaining contextual relevance (deviating from the reference model)(Appendix .4).

We perform the following experiments to verify this assumption. We train **one more epoch** during the SFT phase, aiming to reinforce the multi-turn dialogue capability (UltraChat200k is a multi-dialogue dataset).

As shown in Table 3, the performance of the 2-turn dialogue has improved compared to the results presented in Table 1.

Table 3. Performance of models on MT-Bench: The SFT model trained for two epochs using the Qwen2-7B-Base model during the SFT phase.

Method	MT-Bench	
	1-turn	2-turn
SFT	7.60	7.20
$\text{CLM}_{\lambda=0.5}$	8.14	**7.48**
$\text{CLM}_{\lambda=1.0}$	**8.56**	7.39
$\text{CLM}_{\lambda=1.5}$	8.48	7.21
$\text{CLM}_{\lambda=2.0}$	8.51	7.06

CLM allows users to customize the alignment strength. This issue occurs when the model struggles to answer a question due to insufficient context understanding after alignment. As shown in Table 4, we can experiment with different λ values during the multi-turn dialogue phase, as the CLM has alignment control capabilities. Our findings indicate that lowering the λ value in the second-turn dialogue helps the model better comprehend the context and follow instructions more effectively.

Table 4. The MT-Bench score on the second turn, using different λ values across the two dialogue phases.

2-turn \ 1-turn	$\lambda = 0.5$	$\lambda = 1.0$	$\lambda = 1.5$	$\lambda = 2.0$
$\lambda = 0.0$	7.24	7.61	7.58	7.39
$\lambda = 0.3$	7.59	7.35	7.60	**7.71**
$\lambda = 0.5$	7.53	7.40	7.40	7.68
$\lambda = 0.7$	7.18	7.45	7.19	7.39

In this case, $0 < \lambda < 1$ may also exhibit the alignment extrapolation phenomenon[1].

[1]This does not necessarily mean that the alignment performance will fall somewhere in between, as the aligned model may exhibit low performance due to excessive reward fitting.

5.3. Performance of Reasoning

In addition, we conduct reasoning tasks to evaluate whether the added layer can learn the preference signal without compromising the foundational capabilities of the LM. As shown in Table 2, reasoning ability decreases slightly as model alignment improves. However, a different trend is observed with TruthfulQA. This is likely because the preference dataset incorporates the honesty property into the model, enhancing its truthfulness.

6. Analysis

6.1. Hyperparameter Stability

We try different β in the DPO algorithm to test our efficient alignment training stability. For β equals 0.01 and 0.1, we use learning rate 5e-6; for β equals 0.001, we use learning rate 5e-7.

Table 5. Analysis of hyperparameter stability

Method	Llama3.2-3B-Base			
	AlpacaEval2		Arena-Hard	MT-Bench
	LC (%)	WR (%)	WR (%)	Score
$\beta = 0.1$	3.82	3.27	4.00	5.89
$\beta = 0.01$	**10.53**	**10.77**	**10.40**	6.44
$\beta = 0.001$	10.48	10.90	10.30	**6.56**
Method	**Llama3.1-8B-Base**			
	AlpacaEval2		Arena-Hard	MT-Bench
	LC (%)	WR (%)	WR (%)	Score
$\beta = 0.1$	7.08	5.47	10.50	6.93
$\beta = 0.01$	17.66	14.61	**25.40**	**7.38**
$\beta = 0.001$	**19.77**	**18.31**	23.20	7.23

As shown in Table 5, using a smaller β yields significant performance improvements. We evaluate our hyperparameter selection at the 3B and 8B scales, observing consistent results across these model sizes. These findings underscore the stability of our efficient alignment method. Moreover, it has been observed that an appropriate β value employed in *CLM* training is also well-suited for full DPO training (Appendix .2).

6.2. Layer Significance

We opt to experiment by freezing the lower layers and fine-tuning the top-k layers, as well as by freezing the upper layers and fine-tuning the bottom-k layers. In this way, we aim to determine which layers are most effective for preference learning, ultimately enhancing LM's alignment capability. The learning rate is 5e-6, and β equals 0.01.

As shown in Table 6, preference learning applied to the top layers of the LM yields limited benefits. In contrast, applying preference learning to the bottom layers significantly enhances the alignment ability of the LM.

Table 6. Analysis of the significance of layers in preference learning

Method	Llama3.2-3B-Base			
	AlpacaEval2		Arena-Hard	MT-Bench
	LC (%)	WR (%)	WR (%)	Score
top-1	3.52	4.03	4.00	5.85
top-3	1.85	2.72	4.70	5.68
bottom-1	**9.49**	9.86	9.80	6.53
bottom-3	9.24	**9.91**	**11.70**	**6.56**
Method	**Llama3.1-8B-Base**			
	AlpacaEval2		Arena-Hard	MT-Bench
	LC (%)	WR (%)	WR (%)	Score
top-1	4.03	4.87	8.20	5.67
top-3	4.05	4.96	10.40	6.32
bottom-1	14.94	13.93	24.80	7.37
bottom-3	**16.38**	**15.51**	**25.30**	**7.40**

6.3. The Function of Added Layer

As shown in Figure 5, the added layer does not transform the original input embeddings into the high dimension, instead mapping the unaligned input embeddings into the aligned ones.

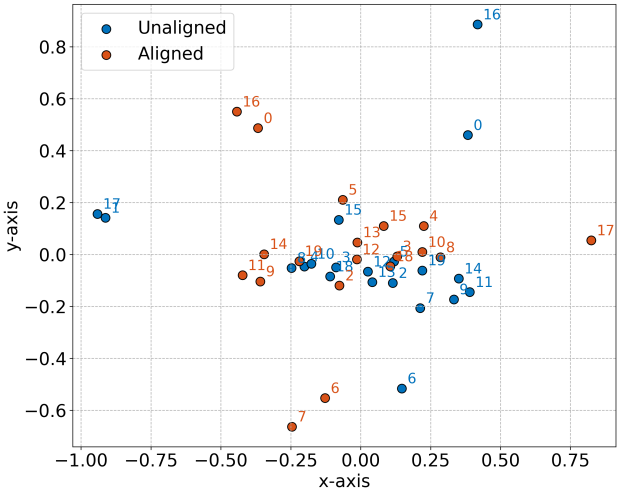

Figure 5. Visualization of input embeddings using the PCA method, with the ID numbers representing token positions.

6.4. The Number of Added Layers

In Analysis 6.2, we know preference learning on the bottom layers would be beneficial. Therefore, we are wondering

if we can add more layers preceding the original layers to improve the alignment ability of LM further. We copy the first layer n times and perform preference learning on these layers.

Table 7. Analysis of the number of added layers

Method	Llama3.2-3B-Base			
	AlpacaEval2		Arena-Hard	MT-Bench
	LC (%)	WR (%)	WR (%)	Score
+1 layer	**10.53**	**10.77**	**10.40**	**6.44**
+2 layers	6.58	7.20	8.3	6.18
+3 layers	7.79	8.81	9.0	6.24
Method	**Llama3.1-8B-Base**			
	AlpacaEval2		Arena-Hard	MT-Bench
	LC (%)	WR (%)	WR (%)	Score
+1 layer	**17.66**	**14.61**	**25.40**	7.38
+2 layers	14.27	12.67	24.70	**7.42**
+3 layers	14.78	13.22	23.40	7.03

As shown in Table 7, increasing the number of added layers does not result in significant performance improvement under the same hyperparameter settings.

6.5. Efficiency Analysis

Our training time and memory costs are significantly reduced compared to fully fine-tuning the model. For instance, we only need to train **2%** of the parameters for LLama3.1-8B. During inference, the weights of all upper layers remain unchanged, effectively overcoming the challenge of requiring two models to be loaded. Moreover, our inference architecture seamlessly integrates with vLLM, albeit with an increased key-value cache.

6.6. Math ability

Table 8. The accuracy on the 32 samples from the MATH500 dataset.

	$\lambda = 0.0$	$\lambda = 0.5$	$\lambda = 1.0$	$\lambda = 1.5$
Acc	37.50	53.13	46.88	40.63

We select the Qwen2.5-1.5B-chat model (Yang et al., 2024a) for this study. A set of 32 samples from the MATH500 dataset (Lightman et al., 2023) is chosen for training, with RL conducted over approximately 770 steps to enable the model to learn from the reward signal. The reward signal employed is sparse and rule-based. After training, the same 32 samples are used to evaluate the model's performance, with the results presented in Table 8. The phenomena of alignment interpolation and extrapolation are also observed.

7. Related Work

Progressive Learning Gong et al. (2019) introduced a stacking approach that incrementally doubles model depth to improve training effectiveness. Expanding on this concept, CompoundGrow (Gu et al., 2020) integrates FeedForward Network expansion into a structured training schedule. More recently, LLama-Pro (Wu et al., 2024) employs depth growth to retain general model performance while enabling adaptation to domain-specific tasks. Our work also employs depth growth at the lowest layer that maps unaligned input embeddings into an aligned space to perform preference learning.

Preference Leaning RLHF is a method aimed at aligning LLMs with human values and preferences (Christiano et al., 2017). The PPO algorithm (Schulman et al., 2017) is frequently employed. However, challenges exist throughout the RLHF process, from collecting preference data to training the model, as highlighted by Radford (2018). Alternatively, techniques like DPO (Rafailov et al., 2024) eliminate the need for a reward model by training LLMs directly based on human preferences. Other competing methods, including IPO (Azar et al., 2024), KTO (Ethayarajh et al., 2024), and WSPO (Zhu et al., 2024), have also emerged.

Realignment The most effective way to achieve realignment is by sweeping the hyperparameter. DeRa (Liu et al., 2024) dynamically adjusts alignment strength at inference time using aligned and unaligned models. Similarly, WSPO (Zhu et al., 2024) demonstrates that when the weak model is identical to the strong model, it can regulate alignment strength during training. We are the first to explore techniques that explicitly incorporate inference-time realignment considerations into the training process.

8. Conclusion

In this paper, we propose *CLM*, an alignment-controllable language model that introduces an identity layer preceding the original layers for preference learning while keeping other layers frozen, achieving efficient training and performance comparable to full fine-tuning. This design enables dynamic alignment control via an interpolation coefficient at inference time.

Future work. See limitation in Appendix .1. We hope our work inspires future research on enabling the conversion of generic models into specialized models in a uniform framework. Furthermore, the current O1-series models may be prone to overthinking issues (Chen et al., 2024), and we believe that introducing alignment control could help alleviate this challenge.